COMMONMARK · GITHUB FLAVORED MARKDOWN · CLAUDE

Guía de Markdown

Cómo escribir texto con formato usando solo caracteres normales: la sintaxis clásica, las extensiones de GitHub y el uso de Markdown con Claude, con el código que se escribe y el resultado que se ve.

```
# Cierre de octubre de 2026
## 1. Resumen
Las ventas ascienden a **1.083.815 €**,
un **+5,2 %** sobre octubre de 2025.

- Online: +16,6 %
- Bilbao: −3,4 %
```

TEXTO PLANO
QUE SE LEE IGUAL
CON O SIN FORMATO

[CONTENIDO]

Índice

En cada apartado, el código que se escribe aparece junto al resultado que produce. El capítulo 9 aplica todo lo anterior a los archivos con los que se configura Claude.

1 ¿Qué es Markdown? 03

2 Sintaxis básica (CommonMark) 05

2.1 Encabezados 05
2.2 Párrafos y saltos de línea 05
2.3 Énfasis: negrita, cursiva y más 06
2.4 Reglas horizontales 07
2.5 Listas 07
2.6 Enlaces 08
2.7 Imágenes 10
2.8 Citas 10
2.9 Código 11
2.10 Escapado de caracteres 11
2.11 HTML dentro de Markdown 12

3 Sintaxis extendida de GitHub (GFM) 13

3.1 Tablas 13
3.2 Texto tachado 14
3.3 Listas de tareas 14
3.4 Autoenlaces, menciones y referencias 15
3.5 Emojis 15
3.6 Notas al pie 16
3.7 Alertas 16
3.8 Secciones plegables 17

4 Contenido avanzado 18

4.1 Matemáticas con LaTeX 18
4.2 Diagramas con Mermaid 19
4.3 Front matter (metadatos YAML) 20

5 Buenas prácticas y errores comunes 21

6 Hoja de trucos 23

7 Editores y herramientas 24

8 Para seguir aprendiendo 25

9 Markdown para Claude CLAUDE 27

9.1 Por qué Claude trabaja en Markdown 27
9.2 Instrucciones y CLAUDE.md 28
9.3 Anatomía de SKILL.md 29
9.4 Plantilla comentada de SKILL.md 31
9.5 El archivo de un agente 32
9.6 Errores típicos 33
9.7 Lista de comprobación final 35

[CAPÍTULO 01]

¿Qué es Markdown?

Un puñado de marcas escritas con el teclado que se transforman en un documento con formato.

Markdown es una forma de dar formato a un texto sin salir del propio texto: unas pocas marcas escritas con el teclado (como `#`, `*`, `-` o `[ ]`) indican qué es un título, qué va en negrita o qué forma parte de una lista, y un programa las convierte después en HTML. Un documento así se visualiza correctamente en cualquier dispositivo, puede almacenarse en sistemas de control de versiones como Git y, aunque nadie lo convierta, se entiende a simple vista.

Lo ideó John Gruber en 2004 con un objetivo concreto: que dar formato a un texto fuera tan natural como escribir un mensaje de correo sin adornos. Para conseguirlo sustituyó las etiquetas extensas, como `<strong>` o `<h1>`, por símbolos discretos que apenas interrumpen la lectura.

[ANALOGÍA]

Markdown funciona como las anotaciones que se hacen a mano sobre un borrador: una doble raya bajo el título, un círculo en torno a lo importante, un guion delante de cada elemento de una enumeración. Quien lee el borrador entiende las marcas sin ayuda, y quien lo maqueta sabe exactamente qué formato aplicar.



[DIAGRAMA 01] EL RECORRIDO DE UN ARCHIVO MARKDOWN: SE ESCRIBE COMO TEXTO, SE INTERPRETA Y SE MUESTRA CON FORMATO

Ventajas principales

[·] **Es texto plano.** Cualquier editor abre un archivo `.md` y su contenido se entiende sin instalar ningún programa específico.

[·] **Es portable.** No está atado a una aplicación ni a un formato binario cerrado.

[·] **Se versiona bien.** Como es texto, Git señala con precisión qué líneas han cambiado entre una versión y otra.

[·] **Se convierte con facilidad** a HTML, PDF, DOCX y a numerosos formatos más.

Dónde se encuentra

Se utiliza en los `README` de los repositorios de GitHub y GitLab, en la documentación técnica, en los blogs estáticos, en aplicaciones de notas como Obsidian o Notion y, de forma muy habitual, en la conversación con asistentes de inteligencia artificial: las respuestas de Claude suelen llegar en Markdown y los archivos con los que se configura a Claude (capítulo 9) también lo son.

CommonMark y GFM

La descripción original de Gruber dejaba algunos casos sin resolver, y por eso la comunidad redactó **CommonMark**, una especificación que establece con exactitud cómo debe interpretarse cada símbolo. En esta guía, «Markdown básico» equivale a CommonMark. GitHub partió de esa misma base y la amplió con funciones propias, conocidas como **GitHub Flavored Markdown (GFM)**, que ocupan el capítulo 3.

[NOTA]
Cada procesador de Markdown tiene sus particularidades: un mismo archivo puede verse de manera distinta en GitHub, en un editor de notas o en la interfaz de Claude. Si el resultado importa, conviene comprobarlo en el entorno final.

Cómo leer esta guía

Cada elemento se presenta en una pareja de bloques: uno oscuro con **lo que escribes**, y otro claro con **lo que se ve**. Cuando el código es largo, el bloque claro pasa debajo. En los bloques oscuros, los símbolos propios de Markdown se resaltan en lima para localizarlos de un vistazo.

LO QUE ESCRIBES

```
Las ventas suben un **5,2 %**
sobre octubre de 2025.
```

LO QUE SE VE

Las ventas suben un **5,2 %** sobre octubre de 2025.

A lo largo de la guía aparecen cinco tipos de aviso: **Nota** (información útil), **Consejo** (una forma mejor de hacer algo), **Importante** (lo imprescindible para lograr el objetivo), **Aviso** (algo que exige atención) y **Precaución** (riesgos de una acción). El apartado 3.7 los muestra todos juntos.

[CAPÍTULO 02]

Sintaxis básica (CommonMark)

Lo necesario para la mayoría de los documentos: títulos, párrafos, énfasis, listas, enlaces, imágenes, citas y código.

2.1 Encabezados

Existen seis niveles de título. Cuantas más almohadillas (`#`) se escriben, más profundo es el nivel, y entre las almohadillas y el texto hay que dejar siempre un espacio.

LO QUE ESCRIBES

```
# Memorándum de cierre
## 1. Resumen
### Ventas por tienda
#### Online
##### Octubre
###### Detalle
```

LO QUE SE VE

Memorándum de cierre
1. Resumen
Ventas por tienda
Online
Octubre
Detalle

Los dos primeros niveles admiten además una forma alternativa: la línea de texto se «subraya» con signos igual (nivel 1) o con guiones (nivel 2).

LO QUE ESCRIBES

```
Memorándum de cierre
====================

1. Resumen
----------
```

LO QUE SE VE

Memorándum de cierre
1. Resumen

[CONSEJO]

Lo habitual es un solo título de nivel 1 por documento y seguir el orden de los niveles sin saltos (tras un `##` viene un `###` , no un `####`). Con esa estructura, los lectores de pantalla navegan mejor y los programas pueden elaborar un índice por su cuenta.

2.2 Párrafos y saltos de línea

Una línea vacía marca el final de un párrafo y el comienzo del siguiente. Si varias líneas se escriben una tras otra, sin ese espacio entre ellas, Markdown las funde en un único párrafo.

Cuando hace falta cortar una línea sin abrir otro párrafo, hay tres opciones: dejar dos espacios al final y pulsar Intro, escribir la etiqueta `<br>` o poner una barra invertida (`\`) como último carácter.

LO QUE ESCRIBES

```
Las ventas de octubre
crecen un 5,2 %.

El margen se mantiene
en el 60,1 %.

Cierre del mes: octubre.··
Envío: lunes 9 de noviembre.
```

LO QUE SE VE

Las ventas de octubre crecen un 5,2 %.

El margen se mantiene en el 60,1 %.

Cierre del mes: octubre.
Envío: lunes 9 de noviembre.

Los puntos medios (`··`) representan los dos espacios del final de la línea, que en el archivo real no se ven.

[AVISO]

Los dos espacios finales son una marca frágil: no se ven, y muchos editores y formateadores los borran al guardar, con lo que el salto se pierde sin ningún aviso. Para que se conserve, es más seguro `<br>` o la barra invertida.

2.3 Énfasis: negrita, cursiva y más

Para destacar una palabra o una frase se la encierra entre asteriscos o entre guiones bajos.

LO QUE ESCRIBES

```
*cursiva* o _cursiva_
**negrita** o __negrita__
***negrita y cursiva***
~~tachado~~  (extensión de GFM)
```

LO QUE SE VE

cursiva o *cursiva*
negrita o **negrita**
negrita y cursiva
~~tachado~~

[CONSEJO]

Ambos símbolos dan el mismo resultado, con una diferencia: el guion bajo no actúa en mitad de una palabra. Por eso `mi_variable` se respeta tal cual, mientras que con asteriscos sí es posible resaltar solo un fragmento (`im*por*tante`). Conviene tenerlo presente al escribir nombres de archivo o de variable.

Dentro del código en línea el formato no se aplica: en `` `**texto**` `` los asteriscos aparecen tal cual. Si se quiere destacar un fragmento de código, el énfasis debe quedar fuera de los acentos graves:

LO QUE ESCRIBES

```
**`git status`** y *`variable`*
```

LO QUE SE VE

`git status` y *`variable`*

El subrayado, el superíndice y el subíndice no cuentan con símbolos propios en Markdown; si el entorno acepta HTML, se resuelven con etiquetas:

LO QUE ESCRIBES

```
Local de 120 m<sup>2</sup> y
CO<sub>2</sub>, con texto
<u>subrayado</u>.
```

LO QUE SE VE

Local de 120 m^2 y CO_2, con texto subrayado.

2.4 Reglas horizontales

Para trazar una línea de separación entre dos bloques basta una línea con tres o más guiones, asteriscos o guiones bajos, y nada más.

LO QUE ESCRIBES

```
---
***
___
```

LO QUE SE VE

Las tres formas trazan la misma línea:

Un fallo habitual es colocar los guiones pegados a la línea de texto anterior:

LO QUE ESCRIBES

```
Total de ventas
---
```

LO QUE SE VE

Total de ventas

SE CONVIERTE EN UN ENCABEZADO DE NIVEL 2

[PRECAUCIÓN]

La regla horizontal necesita una línea vacía por encima. Sin ella, Markdown entiende los guiones como el subrayado de un título de nivel 2 y el texto anterior se convierte en encabezado.

2.5 Listas

Hay listas de viñetas (no ordenadas) y listas numeradas (ordenadas). Cada viñeta se abre con `-`, `*` o `+` y un espacio. Las sublistas se consiguen sangrando el elemento dos o cuatro espacios, siempre con el mismo criterio a lo largo del documento.

LO QUE ESCRIBES

```
- Ventas por tienda
- Margen bruto
- Tesorería
  - Conciliación bancaria
  - Partidas en tránsito
    - Cobros del TPV online
- Incidencias
```

LO QUE SE VE

- Ventas por tienda
- Margen bruto
- Tesorería
  - Conciliación bancaria
  - Partidas en tránsito
    - Cobros del TPV online
- Incidencias

En las listas numeradas, cada elemento lleva un número, un punto y un espacio. Solo importa el primer número, que fija el comienzo de la cuenta (empezar en `3.` produce 3, 4, 5…); el procesador renumera el resto por su cuenta. De ahí la costumbre de escribir `1.` delante de todos los elementos y dejar que la numeración se genere sola.

LO QUE ESCRIBES

```
1. Descargar el extracto
2. Conciliar con el mayor
   1. Marcar lo coincidente
   2. Listar lo pendiente
3. Redactar el memorándum
```

LO QUE SE VE

1. Descargar el extracto
2. Conciliar con el mayor
    1. Marcar lo coincidente
    2. Listar lo pendiente
3. Redactar el memorándum

Dentro de un elemento de lista también caben párrafos, bloques de código o citas, siempre que estén bien sangrados: el contenido debe empezar en la misma columna que el texto del elemento, y no que su marcador. Eso supone dos espacios después de un `-` y tres después de `1.`. Con menos sangría, el contenido se escapa de la lista o la numeración se rompe.

LO QUE ESCRIBES

````
1. Abrir la carpeta de cierre.
   Este párrafo es del paso 1
   (sangría de tres espacios).
   ```text
   cierre-octubre/
   ```
2. Pedir la conciliación.
````

LO QUE SE VE

1. Abrir la carpeta de cierre.
   Este párrafo es del paso 1 (sangría de tres espacios).
   ```
   cierre-octubre/
   ```
2. Pedir la conciliación.

2.6 Enlaces

Un enlace puede escribirse de tres maneras: con un texto visible, con un texto y un título que aparece al pasar el cursor, o con la dirección entre ángulos, que se muestra tal cual.

LO QUE ESCRIBES

```
[Normas de cierre](https://example.com/normas)
[Normas](https://example.com/normas "Documento interno")
<https://example.com/normas>
```

LO QUE SE VE

Normas de cierre
Normas
https://example.com/normas

Enlaces de referencia. Cuando una dirección es muy larga o se usa varias veces, se declara una única vez con una etiqueta y se cita por ella en el texto, que así queda más ligero:

LO QUE ESCRIBES

```
Consulta la [documentación oficial][docs] y la [especificación][spec].

[docs]: https://example.com/docs
[spec]: https://example.com/spec
```

LO QUE SE VE

Consulta la documentación oficial y la especificación.

Enlaces internos (anclas). También es posible enlazar a un título del propio documento. GitHub construye su ancla con estas reglas: (1) pasa el título a minúsculas; (2) cambia los espacios por guiones; (3) suprime los signos de puntuación, pero respeta las letras, con tilde y eñe incluidas; y (4) si dos títulos darían la misma ancla, añade `-1`, `-2` y así sucesivamente al final.

LO QUE ESCRIBES

```
Véase el apartado [Tablas](#tablas).
Y el capítulo [¿Qué es Markdown?](#qué-es-markdown).
```

LO QUE SE VE

Véase el apartado Tablas.
Y el capítulo ¿Qué es Markdown?.

[CONSEJO]

Las anclas varían de un procesador a otro. En GitHub lo más práctico es situar el cursor sobre el título y copiar el icono de enlace que se muestra junto a él, en lugar de intentar adivinar la dirección.

2.7 Imágenes

Una imagen se escribe igual que un enlace, con un signo de exclamación (`!`) delante. Lo que va entre corchetes es el texto alternativo: lo leen los lectores de pantalla y aparece si la imagen no se carga. La ruta puede ser completa o relativa a la ubicación del documento. Si la imagen se coloca dentro de la sintaxis de un enlace, funciona como botón.

LO QUE ESCRIBES

```
![Gráfico de ventas por tienda]
(img/ventas.png "Ventas de octubre")

[![Gráfico de ventas](img/ventas.png)]
(https://example.com)
```

LO QUE SE VE

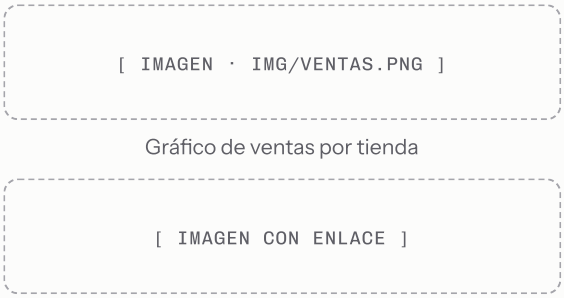


Gráfico de ventas por tienda

[CONSEJO]

Markdown no ofrece forma de fijar el tamaño de una imagen. Cuando hace falta indicar el ancho, se emplea HTML: `<img src="img/ventas.png" alt="Descripción" width="300">` .

2.8 Citas

Una cita es un bloque de texto que se separa visualmente del resto. Se obtiene empezando cada línea con `>` ; si el símbolo se repite, la cita queda anidada dentro de otra. Admite además cualquier otro elemento, como el énfasis o las listas.

LO QUE ESCRIBES

```
> Las ventas crecen un 5,2 %.
>
> > Online sube un 16,6 %.

> **Atención:** Bilbao cae.
>
> - Ventas: 97.912 €
> - Variación: −3,4 %
```

LO QUE SE VE

> Las ventas crecen un 5,2 %.
>
> > Online sube un 16,6 %.

> **Atención:** Bilbao cae.
>
> - Ventas: 97.912 €
> - Variación: −3,4 %

2.9 Código

Hay dos modalidades de código: el fragmento corto, que va dentro de una frase, y el bloque de varias líneas. El fragmento se encierra entre acentos graves y, si él mismo contiene un acento grave, entre dos.

LO QUE ESCRIBES

```
El archivo es `normas-cierre.docx`.

Con dobles: `` el `acento` se respeta ``
```

LO QUE SE VE

El archivo es `normas-cierre.docx`.

Con dobles: `el `acento` se respeta`

Para los bloques conviene la «cerca» de tres acentos graves, que se abre y se cierra en líneas propias. Si tras la apertura se escribe el nombre del lenguaje, el procesador puede colorear la sintaxis.

LO QUE ESCRIBES

````
```python
def margen(ventas, coste):
    return (ventas - coste) / ventas
```
````

LO QUE SE VE

```python
def margen(ventas, coste):
    return (ventas - coste) / ventas
```

[NOTA]

También existe el bloque por sangría, con cuatro espacios al principio de cada línea. Es válido, pero la cerca resulta más clara y admite indicar el lenguaje, de modo que se prefiere en la práctica.

Tres acentos graves dentro de un bloque. Para escribir una cerca dentro de otra (algo habitual al explicar Markdown con ejemplos), la cerca exterior se abre y se cierra con cuatro acentos graves, o más. Basta con que tenga más que cualquiera de las que encierra.

LO QUE ESCRIBES

````
````markdown
```js
console.log("bloque dentro de otro");
```
````
````

LO QUE SE VE

````
```js
console.log("bloque dentro de otro");
```
````

2.10 Escapado de caracteres

A veces un símbolo de Markdown debe aparecer como símbolo y no como instrucción: un asterisco, una almohadilla, un guion bajo. Para ello se coloca una barra invertida (`\`) justo antes, y el símbolo pierde su significado especial.

LO QUE ESCRIBES

```
\# Pendiente de revisar
\*Aviso\*: fecha provisional
```

LO QUE SE VE

Pendiente de revisar
Aviso: fecha provisional

CommonMark permite escapar cualquier signo de puntuación ASCII. En el uso diario, los más frecuentes son los siguientes:

CARACTERES QUE SE ESCAPAN CON MÁS FRECUENCIA

\ ` * _ { } [ ] ( ) # + - . ! | $

[CONSEJO]
En GitHub, escribir `\$` evita que un importe como `$5` se interprete como el inicio de una fórmula matemática (véase el apartado 4.1).

2.11 HTML dentro de Markdown

Markdown admite HTML escrito directamente dentro del texto, y eso amplía mucho sus posibilidades. Cuando falta algo (un vídeo incrustado, una tabla compleja, un estilo particular), se escribe la etiqueta correspondiente y casi todos los procesadores la respetan.

LO QUE ESCRIBES

```
<p align="center">
  Este párrafo está
  <strong>centrado</strong>
  mediante HTML.
</p>
```

LO QUE SE VE

Este párrafo está **centrado** mediante HTML.

[AVISO]
No todos los entornos permiten HTML libre. Por motivos de seguridad, plataformas como GitHub eliminan etiquetas como `<script>` y algunos atributos. Si no se conoce el entorno de destino, es mejor no depender de HTML avanzado.

Comentarios. Markdown carece de una marca de comentario, pero un comentario de HTML cumple esa función: permite dejar anotaciones para quien edite el archivo o apartar un párrafo sin borrarlo. En el resultado final no se ve.

LO QUE ESCRIBES

```
<!-- Revisar cifras con
Tesorería antes de enviar -->
```

LO QUE SE VE

NO SE MUESTRA NADA

[CAPÍTULO 03]

Sintaxis extendida de GitHub (GFM)

Tablas, tachado, listas de tareas, menciones, notas al pie, alertas y secciones plegables.

La sintaxis básica no lo cubre todo. Para llenar los huecos, GitHub desarrolló **GitHub Flavored Markdown (GFM)**, una ampliación de CommonMark pensada para el trabajo cotidiano. Otras plataformas, como GitLab o los editores de notas, han incorporado buena parte de ella.

3.1 Tablas

Cada columna se delimita con una barra vertical (`|`) y, debajo de la cabecera, se escribe una fila de guiones que la separa del cuerpo. Los dos puntos de esa fila controlan la alineación: `:---` a la izquierda, `:---:` centrada y `---:` a la derecha.

LO QUE ESCRIBES

```
| Tienda   | Centro | Ventas (€) |
| :------- | :----: | ---------: |
| Madrid   |  T03   |    173.199 |
| Online   |  T08   |    167.705 |
| Bilbao   |  T05   |     97.912 |
```

LO QUE SE VE

Tienda	Centro	Ventas (€)
Madrid	T03	173.199
Online	T08	167.705
Bilbao	T05	97.912

[CONSEJO]

El código de una tabla no necesita estar alineado para que funcione; igualar el ancho de las columnas solo mejora la lectura del archivo. Casi cualquier editor puede ajustarlas por sí solo, de modo que no merece la pena hacerlo a mano.

Límites de las celdas. Si una celda debe contener una barra vertical, se escribe `\|`. Tampoco admiten saltos de línea auténticos ni bloques de código con cerca; para varias líneas se recurre a `<br>`.

LO QUE ESCRIBES

```
| Concepto       | Tratamiento        |
| :------------- | :----------------- |
| `4.2 \| 4.3`    | Ajuste o tránsito   |
| Limpieza (sep.) | Se imputa<br>a septiembre |
```

LO QUE SE VE

Concepto	Tratamiento
`4.2 | 4.3`	Ajuste o tránsito
Limpieza (sep.)	Se imputa a septiembre

3.2 Texto tachado

El texto tachado se consigue encerrándolo entre dobles virgulillas (`~~`). Sirve para señalar una cifra o una frase que ha sido sustituida sin eliminar lo anterior.

LO QUE ESCRIBES

```
Saldo contable:
~~373.614,27 €~~ 379.816,77 €
tras los ajustes.
```

LO QUE SE VE

Saldo contable: ~~373.614,27 €~~ 379.816,77 € tras los ajustes.

3.3 Listas de tareas

Sirven para seguir el estado de las tareas. Se escriben como una lista corriente cuyos elementos comienzan con `[ ]` cuando la tarea está pendiente o con `[x]` cuando está hecha.

LO QUE ESCRIBES

```
- [x] Descargar el extracto
- [ ] Conciliar con el mayor
- [ ] Redactar el memorándum
```

LO QUE SE VE

- [x] Descargar el extracto
- [ ] Conciliar con el mayor
- [ ] Redactar el memorándum

[NOTA]

Las casillas son una función de GFM. GitHub y GitLab las muestran como casillas en las que se puede hacer clic; un procesador que solo siga CommonMark, o ciertos editores, enseñan el texto sin transformar. Con las notas al pie, las alertas y los emojis en forma de *shortcode* de este capítulo sucede lo mismo: su resultado depende del procesador.

3.4 Autoenlaces, menciones y referencias

GFM incorpora atajos que, en GitHub, enlazan por sí solos con personas y con elementos del repositorio:

- [·] `@usuario` cita a una persona o a una organización y le envía un aviso.
- [·] `#123` lleva a la incidencia o a la solicitud de cambios con ese número dentro del mismo repositorio.
- [·] `propietario/repo#123` apunta a un elemento de otro repositorio.

Las direcciones web escritas sin más se transforman en enlaces, sin necesidad de la sintaxis `[ ]( )`.

LO QUE ESCRIBES

```
Revisado por @tesoreria;
véanse #42 y
finanzas/cierre#7.
Detalle en
https://example.com.
```

LO QUE SE VE

Revisado por @tesoreria; véanse #42 y finanzas/cierre#7. Detalle en https://example.com.

3.5 Emojis

Un emoji puede escribirse con su *shortcode*, que es su nombre entre dos signos de dos puntos, o pegando el propio carácter Unicode. Este apartado se limita a explicar el mecanismo; en un documento profesional conviene emplearlos con mucha moderación.

LO QUE ESCRIBES

```
Informe listo :rocket:
```

LO QUE SE VE

Informe listo 🚀

[CONSEJO]

Un *shortcode* como `:rocket:` solo se transforma donde existe el diccionario correspondiente (GitHub, GitLab, Slack…). El carácter Unicode, en cambio, se muestra igual en casi cualquier sitio, de modo que es la opción más portable.

3.6 Notas al pie

Una nota al pie añade una aclaración o una fuente sin cortar la lectura. Se compone de una marca dentro del texto y de su definición, que suele situarse al final del archivo.

LO QUE ESCRIBES

```
El margen bruto se mantiene en el 60,1 %.[^1]

[^1]: Margen bruto sobre ventas, frente a octubre de 2025.
```

LO QUE SE VE

El margen bruto se mantiene en el 60,1 %.[1]

[1] Margen bruto sobre ventas, frente a octubre de 2025.

[NOTA]

Aunque la definición esté escrita en mitad del documento, GitHub traslada la nota al final y añade un pequeño enlace para regresar al punto donde se citó. El lugar donde se escriba la definición en el código fuente no cambia el lugar en que se muestra.

3.7 Alertas

Una alerta destaca un mensaje dentro del texto. En GitHub es una cita cuya primera línea lleva un marcador especial; cada alerta es una cita independiente y se separa de la siguiente con una línea en blanco. Hay cinco tipos y su aspecto depende de la plataforma: los avisos de esta guía son una versión propia de ellas.

LO QUE ESCRIBES

```
> [!NOTE]
> Información de contexto.

> [!TIP]
> Una forma mejor de hacerlo.

> [!IMPORTANT]
> Información clave.

> [!WARNING]
> Algo que exige atención.

> [!CAUTION]
> Riesgos de una acción.
```

LO QUE SE VE

[NOTA] Información de contexto.

[CONSEJO] Una forma mejor de hacerlo.

[IMPORTANTE] Información clave.

[AVISO] Algo que exige atención.

[PRECAUCIÓN] Riesgos de una acción.

3.8 Secciones plegables

La etiqueta HTML `<details>` produce un bloque que se despliega y se pliega con un clic, adecuado para esconder contenido extenso: registros, soluciones o detalles opcionales.

LO QUE ESCRIBES

```
<details>
<summary>Ver el detalle</summary>

El contenido oculto puede
escribirse con **Markdown**:

- Partida en tránsito
- Ajuste de comisión

</details>
```

LO QUE SE VE

<details open>
<summary>Ver el detalle</summary>

El contenido oculto puede escribirse con **Markdown**:

- Partida en tránsito
- Ajuste de comisión

</details>

[NOTA]

Tras `</summary>` debe haber una línea vacía; si no, el Markdown del interior no se procesa bien.

[CAPÍTULO 04]

Contenido avanzado

Fórmulas, diagramas y metadatos: contenido enriquecido que depende del procesador.

Algunas plataformas amplían Markdown con elementos que van más allá del texto. Los tres de este capítulo no pertenecen a CommonMark, por lo que conviene confirmar que el entorno de destino los admite antes de apoyarse en ellos.

4.1 Matemáticas con LaTeX

Varios procesadores, entre ellos GitHub, representan fórmulas escritas en LaTeX. Una fórmula dentro de una frase va entre dos signos de dólar (`$`); una fórmula en su propia línea, entre dobles (`$$`).

LO QUE ESCRIBES

```
El margen bruto es $M = V - C$ y su
porcentaje se calcula así:

$$
\text{Margen} =
\frac{V - C}{V} \times 100
$$
```

LO QUE SE VE

El margen bruto es $M = V - C$ y su porcentaje se calcula así:

$$\text{Margen} = \frac{V - C}{V} \times 100$$

[PRECAUCIÓN]

Como el signo de dólar delimita las fórmulas, un texto como `importe de $5 y $10` puede interpretarse como una fórmula y mostrarse deformado. Si el documento contiene importes en esa divisa, hay que escapar el signo (`\$5`).

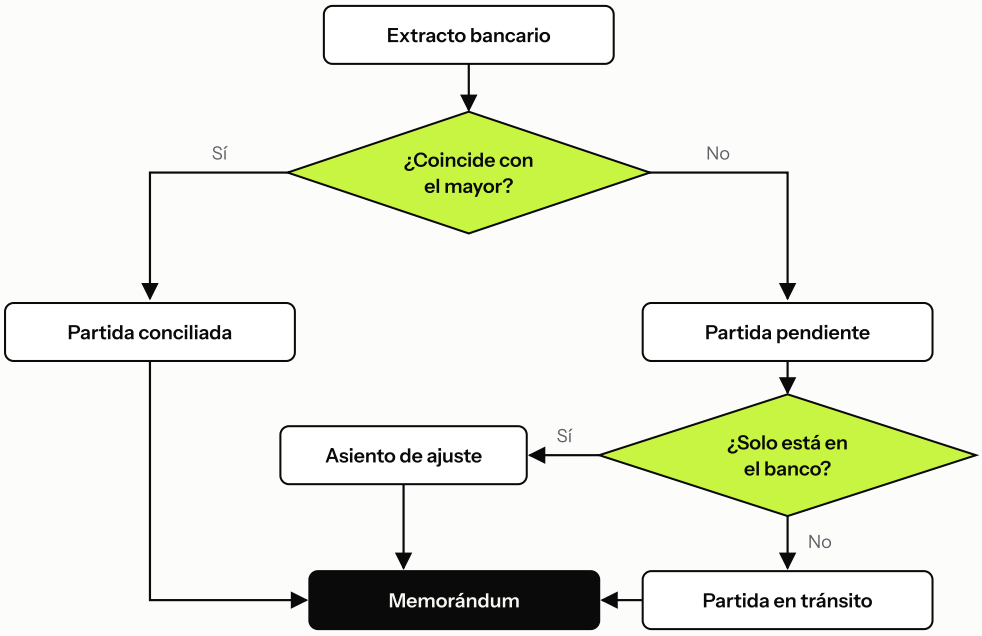
4.2 Diagramas con Mermaid

Mermaid es una biblioteca que dibuja diagramas a partir de una descripción escrita; GitHub y otras plataformas la incluyen. La descripción se coloca en un bloque con cerca cuyo lenguaje es `mermaid`. El ejemplo recoge el criterio de conciliación del caso práctico del módulo: qué ocurre con cada partida del extracto.

LO QUE ESCRIBES

````
```mermaid
flowchart TD
    A[Extracto bancario] --> B{¿Coincide con el mayor?}
    B -->|Sí| C[Partida conciliada]
    B -->|No| D[Partida pendiente]
    D --> E{¿Solo está en el banco?}
    E -->|Sí| F[Asiento de ajuste]
    E -->|No| G[Partida en tránsito]
    C --> H[Memorándum]
    F --> H
    G --> H
```
````

LO QUE SE VE



Además del diagrama de flujo, Mermaid dibuja diagramas entidad-relación, de estados, de clases, de Gantt y de secuencia, entre otros. El aspecto exacto depende del procesador; el dibujo de esta página reproduce el mismo flujo con el estilo de la guía.

4.3 Front matter (metadatos YAML)

El *front matter* es un bloque de metadatos escrito en YAML, entre dos líneas de tres guiones, que se coloca al principio del archivo. Lo leen generadores de sitios como Jekyll, Hugo o Astro y muchos gestores de notas; no forma parte del texto visible y sirve para registrar el título, la fecha, las etiquetas u otros datos del documento.

LO QUE ESCRIBES

```
---
titulo: Memorándum de cierre
fecha: 2026-11-09
etiquetas: [finanzas, cierre]
borrador: false
---
# El contenido empieza aquí
```

LO QUE SE VE

El contenido empieza aquí

EL BLOQUE DE METADATOS NO SE MUESTRA

[IMPORTANTE]

El front matter es el mismo mecanismo que usan los archivos `SKILL.md` y los agentes de Claude, que lo leen para saber cómo se llama cada pieza y cuándo debe usarse. El capítulo 9 explica sus reglas y sus errores más frecuentes.

[CAPÍTULO 05]

Buenas prácticas y errores comunes

Hábitos que previenen los fallos más habituales y un repertorio de errores con su causa y su corrección.

5.1 Hábitos recomendados

[01]

Una oración por línea

En los archivos que se versionan con Git, escribir una oración en cada línea hace que los cambios se lean mejor al compararlos. Como Markdown junta las líneas contiguas de un párrafo, el resultado visible no varía.

[02]

Espacio alrededor de los bloques

Los títulos, las listas, las citas, las tablas y los bloques de código funcionan con más fiabilidad cuando tienen una línea vacía por encima y otra por debajo.

[03]

Sangría uniforme

Se elige una medida, de dos o de cuatro espacios, y se mantiene en todo el archivo. Los tabuladores se evitan porque cada procesador los trata de un modo distinto.

[04]

Texto alternativo en todas las imágenes

La descripción va entre los corchetes de `![ ]`: ayuda a quien usa un lector de pantalla y se muestra cuando el archivo gráfico no se encuentra.

[05]

HTML solo cuando es imprescindible

Se reserva para lo que Markdown no puede expresar, sabiendo que ciertos entornos lo descartan.

[06]

Símbolos que se interpretan sin querer

Cuando un `*`, un `_` o un `#` no se ve como se esperaba, lo normal es que el procesador lo haya tomado por una instrucción; una barra invertida (`\`) lo corrige.

[07]

Comprobar en el destino

Un mismo archivo se muestra de forma diferente según dónde se abra: GitHub, un editor o una página web. Antes de darlo por bueno, se revisa en el sitio donde se va a leer.

5.2 Errores frecuentes y su causa

La mayoría de los fallos de formato se reconocen por su síntoma. Esta tabla permite ir del efecto visible a su causa más probable y a la corrección.

SÍNTOMA	CAUSA PROBABLE	CORRECCIÓN
El título no se convierte en encabezado	Falta el espacio tras las almohadillas: `#Título`	Escribir `# Título`
La lista aparece como un párrafo corrido	Falta la línea en blanco previa a la lista	Dejar una línea en blanco antes y después
Aparece cursiva donde no debía	Asteriscos sueltos en el texto, como en `3*5*2`	Escapar con barra invertida (`3\*5\*2`) o usar código en línea
La tabla se ve como texto con barras	Falta la línea de guiones bajo la cabecera, o la línea en blanco previa	Añadir `| --- | --- |` y separar la tabla del párrafo anterior
El texto de arriba se convierte en título	Una línea de guiones o de signos igual pegada al texto	Dejar una línea en blanco antes de `---`
El contenido anidado sale de la lista	Sangría insuficiente	Alinear con el texto del elemento: dos espacios tras `-` , tres tras `1.`
El salto de línea ha desaparecido	Los dos espacios finales se eliminaron al guardar	Usar ` ` o una barra invertida al final de la línea
La imagen no se ve	Ruta incorrecta respecto al documento	Revisar la ruta y comprobar que el archivo existe
Un importe con `$` se deforma	Se interpreta como fórmula LaTeX	Escapar el signo: `\$5`

[CAPÍTULO 06]

Hoja de trucos

La sintaxis completa en dos tablas, para consultarla de un vistazo.

CLÁSICO (COMMONMARK)

ELEMENTO	SINTAXIS
Encabezados	`# H1` … `###### H6`
Negrita	`**texto**`
Cursiva	`*texto*`
Negrita y cursiva	`***texto***`
Regla horizontal	`---` · `***` · `___`
Lista no ordenada	`- elemento` (o `*` / `+`)
Lista ordenada	`1. elemento`
Enlace	`[texto](url)`
Enlace automático	`<url>`
Imagen	`![alt](url)`
Cita	`> texto`
Código en línea	`` `código` ``
Bloque de código	` ```lenguaje ` … ` ``` `
Escape	`\*` · `\#` · …
HTML	Se incrusta directamente
Comentario	`<!-- no se muestra -->`
Salto de línea	Dos espacios al final, ` ` o `\`

GITHUB (GFM) Y CONTENIDO AVANZADO

ELEMENTO	SINTAXIS
Tabla	`| Col | Col |` + `| --- | --- |`
Alineación de tabla	`:---` izquierda · `:---:` centro · `---:` derecha
Tachado	`~~texto~~`
Lista de tareas	`- [x] hecha` · `- [ ] pendiente`
Mención	`@usuario`
Referencia	`#123` · `propietario/repo#123`
Emoji	`:rocket:` o el carácter Unicode
Nota al pie	`texto[^1]` + `[^1]: definición`
Alerta	`> [!NOTE]` · `TIP` · `IMPORTANT` · `WARNING` · `CAUTION`
Sección plegable	`<details><summary>…</summary>…</details>`
Matemáticas	`$en línea$` · `$$en bloque$$`
Diagrama	` ```mermaid ` … ` ``` `
Front matter	`---` + metadatos YAML + `---` al comienzo del archivo

[CAPÍTULO 07]

Editores y herramientas

Markdown puede escribirse en cualquier editor de texto; estas herramientas añaden vista previa y comodidades específicas.

HERRAMIENTA	QUÉ OFRECE	CUÁNDO CONVIENE
Visual Studio Code	Muestra la vista previa dentro del propio editor (`Ctrl/Cmd + Shift + V`) y se amplía con extensiones de revisión y formato. code.visualstudio.com	Documentación de proyectos y archivos de configuración, en un entorno técnico.
Obsidian	Organiza notas guardadas como archivos `.md` y permite enlazar unas con otras. obsidian.md	Una base de conocimiento con muchas notas relacionadas entre sí.
Typora	Editor minimalista que muestra el formato mientras se escribe, sin separar el código de la vista previa. typora.io	Quien prefiere ver el resultado y no los símbolos.
iA Writer	Editor de interfaz muy sobria, pensado para escribir sin interrupciones. ia.net/writer	Textos largos que exigen concentración.
StackEdit y Dillinger	Funcionan desde el navegador, sin instalar ningún programa.	Pruebas rápidas, o equipos en los que no se pueden instalar programas.

Validación y formato automáticos

Para revisar y dar formato de manera automática existen herramientas como **markdownlint** y **Prettier**, que mantienen un estilo uniforme cuando varias personas escriben en el mismo proyecto. Controlan, por ejemplo, la sangría, las líneas en blanco alrededor de los bloques y la jerarquía de los encabezados. También puede pedirse a Claude que revise un archivo Markdown y señale los errores de sintaxis o de estructura; el capítulo 9 explica qué archivos de Claude están escritos en este formato.

Cómo elegir

- [·] Si solo se va a escribir de vez en cuando, un editor en el navegador es suficiente.
- [·] Si se trabaja con muchas notas enlazadas entre sí, un gestor de notas como Obsidian resulta más cómodo.
- [·] Si hay proyectos con control de versiones, un editor de código con un validador evita errores antes de publicar.

[CAPÍTULO 08]

Para seguir aprendiendo

Las fuentes de referencia y un ejercicio de repaso antes de pasar a los archivos de Claude.

Especificación de CommonMark REFERENCIA	Define con precisión el comportamiento de la sintaxis básica. commonmark.org
Tutorial interactivo PRÁCTICA	Ejercicios breves en el navegador para comprobar cada elemento. commonmark.org/help
Guía de GitHub REFERENCIA	Escritura y formato en GitHub: GFM, alertas, notas al pie y más. docs.github.com/get-started/writing-on-github
Documentación de Mermaid REFERENCIA	Tipos de diagrama y su sintaxis. mermaid.js.org

Ejercicio de repaso

Este texto está en plano. Se propone convertirlo a Markdown: un título de nivel 1, las cifras en negrita, una lista con las tres tiendas que más crecen, una cita con la tienda que cae y una tabla con las variaciones de Online y de Bilbao.

[TEXTO DE PARTIDA]

Resumen de octubre. Ventas: 1.083.815 €, un 5,2 % más que en octubre de 2025. Las tiendas que más crecen son Online (+16,6 %), Lisboa (+14,6 %) y Sevilla (+5,1 %). La única tienda que cae es Bilbao (−3,4 %).

Solución

LO QUE ESCRIBES

```
# Resumen de octubre

Ventas: **1.083.815 €**, un
**5,2 %** más que en octubre
de 2025.

Tiendas que más crecen:

- Online (+16,6 %)
- Lisboa (+14,6 %)
- Sevilla (+5,1 %)

> La única tienda que cae es
> Bilbao.

| Tienda | Variación |
| :----- | --------: |
| Online |  +16,6 % |
| Bilbao |   −3,4 % |
```

LO QUE SE VE

Resumen de octubre

Ventas: **1.083.815 €**, un **5,2 %** más que en octubre de 2025.

Tiendas que más crecen:

- Online (+16,6 %)
- Lisboa (+14,6 %)
- Sevilla (+5,1 %)

> La única tienda que cae es Bilbao.

Tienda	Variación
Online	+16,6 %
Bilbao	−3,4 %

[CAPÍTULO 09]

Markdown para Claude

Por qué Claude trabaja en Markdown y cómo se escriben los archivos con los que se le configura: instrucciones, skills y agentes.

9.1 Por qué Claude trabaja en Markdown

Markdown es el idioma común entre las personas y Claude por tres razones. La primera es que las respuestas de Claude suelen llegar en Markdown (títulos, listas, tablas y bloques de código) y la interfaz las maqueta; escribir en Markdown permite, a la inversa, indicar con precisión la estructura que se espera. La segunda, que un texto con estructura explícita suele ser más fácil de seguir, para una persona y para un modelo, que un párrafo sin marcas. La tercera, que las piezas con las que se configura a Claude son archivos de texto en Markdown, muchas veces con una cabecera YAML, y por eso pueden leerse, revisarse, compartirse y versionarse como cualquier otro documento.

[ANALOGÍA]

Cuando un colaborador se incorpora a la empresa recibe tres cosas por escrito: las normas de la casa, los manuales de cada procedimiento y la ficha de su puesto. Con Claude ocurre algo parecido: `CLAUDE.md` son las normas de la casa, `SKILL.md` es el manual de un procedimiento y el archivo de un agente es la ficha de un especialista.

ARCHIVO	PARA QUÉ SIRVE	DÓNDE VIVE
`CLAUDE.md`	Las normas de la casa: instrucciones que Claude Code lee al empezar cada sesión.	`./CLAUDE.md` en el proyecto o `~/.claude/CLAUDE.md` en la carpeta del usuario.
`AGENTS.md`	Alternativa a `CLAUDE.md`: si este no existe, Claude Code lee `AGENTS.md`.	En el proyecto.
`SKILL.md`	El manual de un procedimiento: qué hace una skill y cómo se ejecuta.	En la carpeta de la skill, que se sube como ZIP en «Personalización» › «Skills» o forma parte de un plugin (`skills/<nombre>/`).
Archivo de agente	La ficha de un especialista: su encargo, sus herramientas y su modelo.	`.claude/agents/`, `~/.claude/agents/` o `agents/` dentro de un plugin.

Pedirle a Claude la estructura que se necesita

No hace falta escribir Markdown para obtenerlo: basta con pedir la estructura. Cuanto más concreta sea la petición, más cerca estará el resultado de lo que se busca.

EJEMPLO DE PETICIÓN — PROMPT

```
Resume las ventas de octubre por tienda en una tabla con las columnas
Tienda, Ventas (€) y Variación, ordenada de mayor a menor variación.
Después, añade tres frases con las conclusiones.
```

9.2 Instrucciones y CLAUDE.md

Las instrucciones son el texto que Claude lee antes de empezar a trabajar. En la app hay varias capas, de lo general a lo concreto: las instrucciones de la cuenta, las «Instrucciones globales» de Cowork, las del proyecto, las de una carpeta y, por último, el prompt de cada tarea. Todas son texto libre y todas se benefician de la misma disciplina: frases cortas, listas y títulos claros.

En Claude Code, la pestaña «Code» de la app destinada a equipos técnicos, el equivalente es el archivo `CLAUDE.md`. Puede haber uno en el proyecto y otro personal en la carpeta del usuario. El comando `/init` genera uno a partir del proyecto y `/memory` permite editarlo.

CLAUDE.MD — EJEMPLO BREVE

```
# Cierre mensual · instrucciones para Claude

## Contexto
Empresa de calzado y marroquinería con siete tiendas y canal online.
Los datos de esta carpeta son sintéticos y sirven para practicar.

## Cómo escribir
- Español de España, registro formal.
- Importes en euros, sin decimales y con punto de millar (1.083.815 €).
- Porcentajes con un decimal y coma decimal (5,2 %).

## Reglas de trabajo
- No modificar los archivos originales: trabajar siempre sobre una copia.
- Señalar cada supuesto y no inventar datos.
- Ante una duda sobre los criterios, consultar `normas-cierre.docx`.
```

[IMPORTANTE]

`CLAUDE.md` se trata como contexto, no como una regla obligatoria: Claude lo tiene presente, pero no se garantiza que lo cumpla al pie de la letra. Lo que deba cumplirse siempre se configura como un *hook*, el mecanismo de Claude Code para ejecutar una acción de forma automática en un momento concreto.

[CONSEJO]

Las instrucciones eficaces son breves, concretas y verificables: «importes en euros sin decimales» puede comprobarse; «sé preciso» no. En la app, estas mismas instrucciones se escriben en los cuadros de texto de la cuenta, de Cowork o del proyecto, sin necesidad de ningún archivo.

9.3 Anatomía de SKILL.md

Una skill es una carpeta con un archivo `SKILL.md` obligatorio y, si hace falta, tres subcarpetas opcionales. El nombre de la carpeta coincide con el campo `name` del archivo.

CARPETA DE UNA SKILL — ESTRUCTURA

```
memorandum-cierre/
  SKILL.md          obligatorio: cabecera YAML + instrucciones
  scripts/          opcional: código que se ejecuta
  references/       opcional: documentos de consulta
  assets/           opcional: plantillas y otros recursos
```

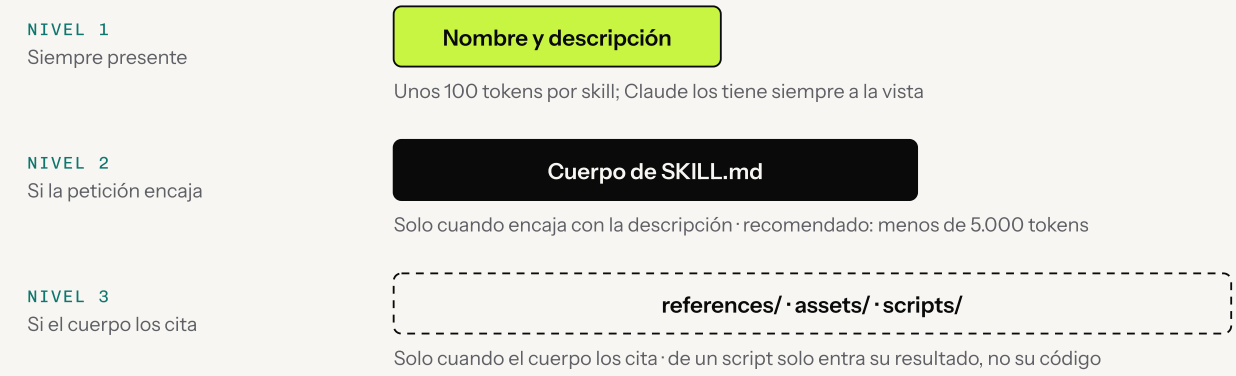
`SKILL.md` tiene dos partes: la cabecera YAML (*front matter*), delimitada por tres guiones, y el cuerpo en Markdown con las instrucciones. La cabecera se rige por estas reglas:

CAMPO	OBLIGATORIO	REGLAS
`name`	Sí	Hasta 64 caracteres: minúsculas, números y guiones. No puede contener «anthropic» ni «claude». Coincide con el nombre de la carpeta.
`description`	Sí	Qué hace la skill y cuándo debe usarse, en tercera persona y con las palabras que emplearía quien la pide. El estándar admite hasta 1.024 caracteres; la ayuda de Claude recomienda no pasar de 200.
`license`	No	La licencia con la que se comparte la skill.
`compatibility`	No	Los requisitos del entorno en el que funciona.
`metadata`	No	Datos adicionales, como el autor o la versión.
`allowed-tools`	No	Las herramientas que la skill puede utilizar.

Lo esencial de YAML

La cabecera está escrita en YAML, un formato de texto muy sensible a la sangría. Para una cabecera de skill bastan cinco reglas:

QUÉ SE QUIERE	CÓMO SE ESCRIBE
Una clave con su valor	`name: memorandum-cierre`
Un texto que contiene dos puntos	Entre comillas: `description: "Cierre: cinco apartados"`
Una lista (en otros archivos YAML)	`tiendas: [Palma, Bilbao]` . La cabecera de una skill no la necesita: solo admite los seis campos de la tabla.
Un nivel de anidación	Dos espacios por nivel, nunca tabuladores
Un comentario	`# texto` , hasta el final de la línea



[DIAGRAMA 02] CARGA PROGRESIVA: CUANTO MÁS ABAJO, MÁS ESPECÍFICO Y MENOS HABITUAL

La carga progresiva explica dos consejos. La descripción debe ser precisa, porque es lo único que Claude ve siempre. Y el cuerpo debe ser breve, porque cada vez que la skill se activa ocupa espacio en el contexto: lo extenso se traslada a `references/` y se cita desde `SKILL.md` a un solo nivel.

9.4 Plantilla comentada de SKILL.md

La plantilla siguiente recoge las partes habituales de una skill. Los comentarios de la cabecera empiezan por `#` y los del cuerpo van entre `<!-- -->`; ninguno de los dos se muestra a quien lee el resultado, y pueden borrarse una vez adaptado el texto.

SKILL.MD PLANTILLA

```
---
# Nombre: igual que la carpeta. Minúsculas, números y guiones.
name: memorandum-cierre
# Descripción: qué hace y cuándo usarla, en tercera persona y con las
# palabras de quien la pide. Mejor por debajo de 200 caracteres.
description: Redacta el memorándum de cierre mensual para Dirección General con los cinco
apartados y el formato de cifras de las normas de cierre. Úsala al preparar o revisar el
cierre de un mes.
---

# Memorándum de cierre

<!-- El cuerpo son las instrucciones: breves, concretas y con ejemplos. -->

## Cuándo usarla
- Cuando se pida redactar o revisar el memorándum de un cierre mensual.

## Entradas
- Las ventas del mes, la conciliación bancaria y las normas de cierre.

## Pasos
1. Leer `references/formato-memorandum.md`.
2. Rellenar los cinco apartados con las cifras reales.
3. Comprobar la fecha de envío frente al calendario de las normas.

## Reglas de formato
- Importes en euros, sin decimales y con punto de millar.
- Porcentajes con un decimal.
- Señalar cada supuesto; no inventar datos.

## Ejemplo
Entrada: «Prepara el memorándum de octubre».
Salida: un documento con los apartados 1 a 5, fecha de envío incluida.

## Qué no hacer
- No modificar los archivos de origen.
```

Para ponerla en marcha, se guarda el archivo como `SKILL.md` dentro de la carpeta de la skill, se comprime esa carpeta en un ZIP (con la carpeta de la skill en el nivel superior del ZIP) y se sube en «Personalización» › «Skills». Después se activa con su interruptor. Subir skills requiere que esté activado el interruptor «Ejecución de código en la nube y creación de archivos», en «Capacidades».

[CONSEJO]

Cuando un proceso deba dar siempre el mismo resultado (cuadrar, convertir, validar), conviene que lo haga un script de la carpeta `scripts/` : solo su resultado entra en el contexto. El texto de `SKILL.md` se reserva para el criterio, el tono y el formato.

9.5 El archivo de un agente

Un agente (o subagente) es un especialista al que el orquestador delega una parte del trabajo. Se define en un archivo `.md` cuyo cuerpo es su prompt de sistema, precedido por una cabecera YAML.

[ANALOGÍA]

Es la ficha de un puesto: el nombre, el cometido, las herramientas a su alcance y el modelo que lo ocupa. El cuerpo del archivo es el procedimiento que seguirá quien lo desempeñe.

CAMPO	OBLIGATORIO	QUÉ CONTIENE
`name`	Sí	El nombre del agente.
`description`	Sí	Qué hace y cuándo conviene delegarle trabajo.
`tools`	No	Lista blanca de las herramientas que puede usar.
`model`	No	El modelo con el que trabaja: `sonnet` , `opus` , `haiku` , `fable` o `inherit` .

REVISOR-CIERRE.MD EJEMPLO

```
---
name: revisor-cierre
description: Revisa un memorándum de cierre mensual contra las normas de cierre (fecha,
apartados, cifras y conciliación) y devuelve una lista de hallazgos. Úsalo antes de dar por
bueno el memorándum.
tools: Read, Grep, Glob, Write
model: sonnet
---

Eres el revisor del cierre mensual. Recibes un memorándum y lo comparas
con `normas-cierre.docx`.

Comprueba, por este orden:

1. La fecha de envío respeta el calendario (D+7; si cae en fin de
   semana, el lunes siguiente).
2. Están los cinco apartados y en el orden previsto.
3. Las cifras usan punto de millar, sin decimales en euros y con
   un decimal en porcentajes.

Devuelve una lista numerada de hallazgos. No corrijas el texto: señala el
problema y el criterio que incumple.
```

Los agentes pueden guardarse en `.claude/agents/` (proyecto), en `~/.claude/agents/` (usuario) o en la carpeta `agents/` de un plugin. Cowork carga los agentes de los plugins y decide por sí mismo cuándo delegar en ellos; en Claude Code se invocan con un control más fino. La conversación en Chat los ignora.

9.6 Errores típicos

Casi todos los fallos de una skill o de un agente se reducen a un puñado de causas. Cada tarjeta muestra el error, su motivo y la forma correcta.

Descripción vaga

La descripción es lo único que Claude ve siempre: si no dice qué hace ni cuándo, la skill no se activa cuando debe o se activa cuando no debe.

MAL

```
description: Ayuda con documentos
```

BIEN

```
description: Redacta el memorándum de
cierre mensual. Se activa cuando se pide el
memorándum o el informe del mes.
```

YAML mal sangrado

En YAML, la sangría indica la jerarquía: sin ella, `autor` deja de pertenecer a `metadata`.

MAL

```
metadata:
autor: Finanzas
```

BIEN

```
metadata:
  autor: Finanzas
```

Tabuladores

YAML no admite tabuladores para sangrar. Hay que usar espacios, dos por nivel, y configurar el editor para que no inserte tabuladores.

MAL
```
[tab]autor: Finanzas
```

BIEN
```
··autor: Finanzas
```

Dos puntos sin comillas

Dos puntos seguidos de un espacio dentro de un valor se leen como el inicio de otro campo y rompen la cabecera. Se evitan o se entrecomilla el valor.

MAL
```
description: Cierre: cinco apartados
```

BIEN
```
description: "Cierre: cinco apartados"
```

Nombre mal formado

Solo minúsculas, números y guiones, hasta 64 caracteres, sin «anthropic» ni «claude», y con el mismo nombre que la carpeta.

MAL
```
name: Memorándum Cierre
```

BIEN
```
name: memorandum-cierre
```

Cuerpo larguísimo

Un cuerpo extenso consume contexto cada vez que la skill se activa. Conviene mantenerlo por debajo de 500 líneas y trasladar el detalle a `references/`.

MAL
```
SKILL.md de 1.200 líneas
```

BIEN
```
SKILL.md de 150 líneas
+ references/kpi.md
```

Fechas que caducan

Una instrucción que depende de un momento concreto deja de ser cierta sin avisar. Es preferible remitir a un documento que se mantenga al día.

MAL
```
Hasta junio se usa el criterio A.
```

BIEN
```
Se usa el criterio de
references/criterios.md.
```

Credenciales en el texto

Una skill, un agente o un `CLAUDE.md` se comparten y se leen: nunca deben contener claves, contraseñas ni tokens.

MAL
```
clave: (el valor real escrito aquí)
```

BIEN
```
Las credenciales las gestiona el conector;
no se escriben en el archivo.
```

ZIP mal empaquetado

Al subir una skill en «Personalización» › «Skills», el nivel superior del ZIP debe ser la carpeta de la skill, no los archivos sueltos.

MAL
```
skill.zip
  SKILL.md
  references/
```

BIEN
```
skill.zip
  memorandum-cierre/
    SKILL.md
    references/
```

9.7 Lista de comprobación final

Antes de subir o compartir una skill, conviene repasar estos puntos uno por uno.

- [ ] El archivo se llama exactamente `SKILL.md` y está dentro de la carpeta de la skill.
- [ ] `name` coincide con el nombre de la carpeta, usa solo minúsculas, números y guiones, y no contiene «anthropic» ni «claude».
- [ ] `description` dice qué hace y cuándo usarla, en tercera persona y con las palabras de quien la pide; mejor por debajo de 200 caracteres.
- [ ] La cabecera se abre y se cierra con `---`, usa espacios (nunca tabuladores) y entrecomilla los valores que llevan dos puntos.
- [ ] El cuerpo tiene menos de 500 líneas; lo extenso está en `references/`, citado desde `SKILL.md` a un solo nivel.
- [ ] La terminología es constante, y no hay fechas que caduquen ni credenciales.
- [ ] Incluye ejemplos de entrada y de salida.
- [ ] Lo que debe dar siempre el mismo resultado (cuadrar, convertir, validar) lo hace un script.
- [ ] Se ha probado con al menos tres casos: uno en el que debe activarse, otro en el que no debe activarse y una invocación explícita con «/». Y con más de un modelo.
- [ ] El ZIP tiene la carpeta de la skill en el nivel superior.

> **«Un texto plano, bien estructurado, es el idioma común entre las personas y Claude.»**
>
> IDEA CLAVE · CAPÍTULO 9